

Planificación de CPU

Análisis utilizando los comandos `time` y `nice`

Sistemas Operativos II

LCC - FCEIA-UNR

8 de abril de 2026

Uso del comando `time` en planificación

- ▶ El comando `time` permite medir el comportamiento real de un proceso:

```
time ./programa
```

- ▶ Devuelve tres métricas:
 - ▶ **real**: tiempo total transcurrido (*turnaround*)
 - ▶ **user**: tiempo en modo usuario
 - ▶ **sys**: tiempo en modo sistema (kernel).
- ▶ Relación con planificación:
 - ▶ CPU usada: `user + sys`
 - ▶ Tiempo de espera (aprox.):

$$\text{waiting} \approx \text{real} - (\text{user} + \text{sys})$$

- ▶ Interpretación:
 - ▶ Mayor `real` $\Rightarrow$ más espera / menor prioridad
 - ▶ Mismo `user` $\Rightarrow$ mismo trabajo computacional

Experimento simple de planificación

- ▶ Ejecutar un proceso utilizando el comando `time` para medir sus tiempos de ejecución:

```
time ./programa
```

- ▶ Ejecutarlo con competencia:

```
./programa & ./otro & time ./programa
```

- ▶ Observar:

- ▶ `real` aumenta significativamente.
- ▶ `user` es similar (no necesariamente igual).

- ▶ Interpretación: El proceso realiza el mismo trabajo computacional, pero pasa más tiempo esperando CPU.

- ▶ **Conclusiones:**

- ▶ El planificador introduce espera (colas, apropiación).
- ▶ Se evidencia el efecto de compartir CPU.

Precaución: sistemas multicore

- ▶ En sistemas modernos hay múltiples núcleos (cores):
 - ▶ Varios procesos pueden ejecutarse en paralelo.
 - ▶ No necesariamente compiten por CPU.
- ▶ El comando `nproc` devuelve el número de CPUs disponibles.
- ▶ Consecuencia:
 - ▶ Ejecutar múltiples procesos **no garantiza** aumento en real.
 - ▶ Puede no observarse el efecto de la planificación.
- ▶ **¿Cómo forzar competencia?**
 - ▶ Lanzar más procesos que la cantidad de núcleos disponibles.
 - ▶ Fijar los procesos al mismo núcleo con `taskset`:

```
taskset -c 0 ./programa & taskset -c 0 ./otro &  
time taskset -c 0 ./programa
```
- ▶ Efecto:
 - ▶ Todos los procesos compiten por un único núcleo.
 - ▶ Aumenta el tiempo de espera (incrementa real).
- ▶ **Conclusión:**
 - ▶ La competencia por el CPU depende del hardware.
 - ▶ Es necesario controlarla para observar el planificador.

Control de prioridad con nice

- ▶ El comando `nice` permite ajustar la prioridad de un proceso:

```
nice -n <valor> ./programa
```

- ▶ Rango de valores:
 - ▶ **-20**: mayor prioridad
 - ▶ **0**: prioridad por defecto
 - ▶ **19**: menor prioridad
- ▶ Ejemplo: `nice -n 10 ./programa`
- ▶ NI es un parámetro configurable que expresa la “amabilidad” del proceso.
- ▶ Mayor valor $\Rightarrow$ menor prioridad efectiva.
- ▶ PR es un parámetro calculado por el sistema que refleja su prioridad efectiva.
- ▶ NI influye en PR, pero este último es dinámico y puede variar durante la ejecución (observable con `top` o `htop`).
- ▶ En Linux (visión simplificada), para procesos normales, el valor mostrado de prioridad se aproxima como: $PR \approx 20 + NI$.

Efecto de nice medido con time

- ▶ Comparar ejecución con distintas prioridades:

```
time nice -n 10 taskset -c 0 ./programa
```

- ▶ Observaciones esperadas:

- ▶ $\text{user} \approx \text{constante}$ (mismo trabajo).
- ▶ real mayor con menor prioridad.

- ▶ Interpretación:

- ▶ Menor prioridad $\Rightarrow$ más tiempo en cola de listos.
- ▶ El planificador asigna menos CPU al proceso.

- ▶ Estimación del tiempo de espera:

$$\text{waiting} \approx \text{real} - (\text{user} + \text{sys})$$

- ▶ **Conclusión:** nice permite ajustar la prioridad relativa de un proceso.

Procesos CPU-bound vs I/O-bound

▶ CPU-bound (intensivo en CPU):

- ▶ Realiza cálculo sin interrupciones.
- ▶ Ejemplo: bucle con muchas operaciones.
- ▶ Resultado típico con `time`:

- ▶ user alto
- ▶ real $\approx$ user

▶ Experimento:

```
taskset -c 0 ./cpu & time taskset -c 0 ./cpu
```

▶ I/O-bound (con bloqueos):

- ▶ Alterna ejecución con esperas (e.g., `sleep`).
- ▶ Resultado típico con `time`:

- ▶ user muy bajo
- ▶ real $\gg$ user
- ▶ real casi no cambia

▶ Experimento:

```
taskset -c 0 ./io & time taskset -c 0 ./io
```

CPU-bound vs I/O-bound en planificación

Conclusiones:

- ▶ Los procesos CPU-bound compiten por CPU:
 - ▶ CPU-bound $\Rightarrow$ alta competencia $\Rightarrow$ aumenta `real`
- ▶ Los procesos I/O-bound liberan CPU al bloquearse:
 - ▶ I/O-bound $\Rightarrow$ baja competencia $\Rightarrow$ `real` casi constante

Errores comunes al usar time

- ▶ **Confundir real con uso de CPU:**
 - ▶ real incluye esperas y bloqueos.
 - ▶ No representa tiempo efectivo de CPU.
- ▶ **Ignorar user y sys:**
 - ▶ Reflejan el trabajo computacional real.
 - ▶ Permiten comparar ejecuciones equivalentes.
- ▶ **No considerar la carga del sistema:**
 - ▶ Otros procesos afectan real.
 - ▶ Resultados pueden variar entre ejecuciones.
- ▶ **Interpretar mal procesos I/O-bound:**
 - ▶ Bajo user no implica poca duración total.
 - ▶ Pueden tener real alto por bloqueos.
- ▶ **No considerar múltiples núcleos:**
 - ▶ En sistemas multicore, los procesos pueden ejecutarse en paralelo.
 - ▶ $\text{user} + \text{sys}$ puede superar a real.
 - ▶ Para experimentos controlados, conviene fijar afinidad (por ejemplo, con taskset).